

RELATÓRIO TÉCNICO

Trabalho 3: Identificação, Classificação e Priorização de Dívida Técnica

Disciplina: DIM0501 – Boas Práticas de Programação

1. Identificação do Grupo

Integrante 1: Randson Thiago Sales da Silva Lima

Linguagem utilizada: C++

Link do repositório Git:

<https://github.com/randon95754/trabalho3>

2. Descrição do Sistema

O sistema analisado consiste em um software simples de controle de estoque e vendas para uma pequena loja. Seu objetivo é permitir o cadastro de produtos, registrar vendas, atualizar automaticamente o estoque, listar os produtos cadastrados e emitir um relatório de produtos com estoque baixo.

O código foi desenvolvido rapidamente para atender uma necessidade imediata, priorizando a entrega do funcionamento básico em detrimento da qualidade interna do software. Como consequência, diversas decisões de implementação foram tomadas sem planejamento adequado, acumulando dívidas técnicas que dificultam a manutenção, a evolução e a confiabilidade do sistema.

O contexto representa uma situação comum no desenvolvimento de software, em que sistemas inicialmente pequenos passam a exigir manutenção contínua e novas funcionalidades. Nesses casos, torna-se necessário identificar quais problemas devem ser corrigidos primeiro, considerando o custo da correção e os riscos de manter essas deficiências.

3. Inventário e Classificação da Dívida Técnica

ID	Localização	Descrição do problema	Tipo	Esforço	Impacto	Juros
D1	Variável global <code>SENHA_ADMIN</code>	Senha administrativa armazenada diretamente no código-fonte (hardcoded).	Configuração	1	5	5
D2	Função <code>main()</code>	Não existe validação das entradas de preço e quantidade. Caso o usuário digite texto, o programa entra em estado de erro.	Robustez	2	5	5
D3	<code>vender()</code> e <code>calcular_total()</code>	A regra de desconto está duplicada e apresenta valores diferentes, podendo gerar cálculos inconsistentes.	Código	2	5	4
D4	Variável global <code>produtos</code>	Toda a aplicação depende de um estado global, aumentando o acoplamento entre funções.	Arquitetura/Design	4	4	4
D5	Função <code>add()</code>	Histórico armazenado em variável estática sem qualquer mecanismo de consulta ou reutilização.	Arquitetura/Design	2	2	2

D6	Função <code>relatorio_vendas()</code>	Método declarado contendo apenas um marcador TODO, sem implementação.	Marcadores/TO DO	2	3	3
D7	Inclusão da biblioteca <code><ctime></code>	Biblioteca importada sem utilização durante a execução do programa.	Código morto	1	1	1
D8	Função <code>exportar()</code>	Código antigo comentado que permanece no projeto sem utilização.	Código morto	1	2	2
D9	Projeto completo	Ausência de testes automatizados para validar o funcionamento das funcionalidades existentes.	Testes	5	5	5
D10	Diversas funções	Uso de nomes pouco descritivos (<code>add</code> , <code>n</code> , <code>p</code> , <code>q</code>), reduzindo a legibilidade.	Código	2	2	3
D11	Diversas funções	Existência de números mágicos (100, 200, 0.10, 0.15 e 5) espalhados pelo código.	Código	2	3	3
D12	Comentários do projeto	Comentários superficiais, desatualizados ou sem valor documental ("feito rápido", "não usado").	Documentação	1	1	1

4. Matriz de Priorização

Ganhos rápidos (alto impacto e baixo esforço)

- D1 – Senha armazenada diretamente no código
- D2 – Falta de validação das entradas
- D3 – Regras de desconto duplicadas

Esses itens apresentam alto impacto sobre a segurança, robustez e corretude do sistema, podendo ser corrigidos com poucas alterações no código.

Grandes apostas (alto impacto e alto esforço)

- D4 – Uso de estado global
- D9 – Ausência de testes automatizados

São problemas importantes, porém exigem refatorações estruturais ou desenvolvimento de uma infraestrutura de testes, demandando maior investimento de tempo.

Tarefas de preenchimento (baixo impacto e baixo esforço)

- D8 – Código comentado
- D10 – Nomes pouco descritivos
- D11 – Números mágicos

São melhorias de qualidade que facilitam futuras manutenções, mas não comprometem diretamente o funcionamento atual do sistema.

Adiar ou aceitar conscientemente

- D5 – Histórico em variável estática
- D6 – Método de relatório ainda não implementado
- D7 – Biblioteca não utilizada
- D12 – Comentários pouco úteis

Esses itens apresentam baixo impacto imediato e podem ser aceitos temporariamente sem comprometer significativamente o funcionamento da aplicação.

Ordem de Prioridade

1. D2 – Falta de validação das entradas
2. D3 – Regras de desconto inconsistentes
3. D1 – Senha hardcoded
4. D9 – Ausência de testes automatizados
5. D4 – Estado global
6. D11 – Números mágicos
7. D10 – Nomes pouco descritivos
8. D8 – Código morto
9. D6 – TODO pendente
10. D5 – Histórico estático
11. D7 – Biblioteca não utilizada
12. D12 – Comentários superficiais

Justificativa

A priorização foi realizada considerando principalmente a relação entre impacto e esforço de correção. Problemas que apresentam elevado risco para o funcionamento do sistema e podem ser resolvidos com pequenas modificações foram classificados como prioridade máxima. Em seguida aparecem os problemas arquiteturais, que possuem grande impacto, porém exigem maior esforço de implementação. Por último permanecem itens relacionados à documentação e organização do código, cujo benefício imediato é menor.

5. Roadmap de Quitação

Pagar agora

D2 – Validação das entradas do usuário

Justificativa:

A ausência de validação permite que qualquer entrada inválida interrompa o funcionamento normal do programa. Corrigir esse problema aumenta significativamente a robustez da aplicação com baixo esforço de implementação.

Critério de pronto:

- O programa continua executando mesmo quando o usuário informa dados inválidos.
- O usuário recebe uma mensagem solicitando nova entrada.

D3 – Unificação da regra de desconto

Justificativa:

Existem duas regras diferentes para cálculo de desconto, o que pode produzir resultados divergentes dependendo da função utilizada. A centralização da regra elimina inconsistências e facilita futuras alterações.

Critério de pronto:

- Apenas uma função realiza o cálculo do desconto.
- Todas as funcionalidades utilizam exatamente a mesma regra de negócio.

Pagar depois

D9 – Implementação de testes automatizados

Embora seja um dos itens mais importantes para garantir a confiabilidade do sistema, sua implementação exige maior tempo e infraestrutura, sendo planejada para uma etapa posterior.

D4 – Eliminação do estado global

A remoção da variável global exigiria uma reorganização significativa da arquitetura da aplicação, sendo mais adequada para uma futura refatoração.

D11 – Remoção dos números mágicos

Os valores constantes podem ser substituídos posteriormente por constantes nomeadas, melhorando a legibilidade sem alterar o comportamento do sistema.

Aceitar conscientemente

Os itens D5, D6, D7, D8, D10 e D12 foram aceitos temporariamente, pois apresentam baixo impacto operacional. Embora representem dívidas técnicas, o custo de corrigi-los neste momento não se justifica diante das prioridades identificadas.

6. Conclusão

A análise realizada permitiu compreender que dívida técnica não representa necessariamente erros de programação, mas sim decisões de desenvolvimento que facilitam a implementação inicial em troca de dificuldades futuras na manutenção do software. Durante o inventário observou-se que o sistema possui problemas relacionados à robustez, arquitetura, documentação, organização do código e segurança.

A etapa de priorização demonstrou que nem toda dívida deve ser eliminada imediatamente. A principal dificuldade foi definir quais problemas realmente justificavam investimento imediato, considerando simultaneamente impacto, esforço e custo de convivência. Essa análise mostrou que decisões baseadas apenas na facilidade de implementação nem sempre produzem o maior benefício para o projeto.

Por fim, conclui-se que o gerenciamento da dívida técnica consiste em identificar, classificar e priorizar adequadamente os problemas existentes, direcionando os esforços para os itens que oferecem maior retorno para a qualidade e a evolução do sistema. A quitação das duas dívidas prioritárias melhora significativamente a robustez e a consistência da aplicação, preservando seu comportamento original e demonstrando a importância de um processo estruturado de manutenção de software.